CHAPTER 20

State Pattern

This chapter covers the State pattern.

GoF Definition

Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

Concept

The concept is best described by the following examples.

Real-Life Example

Consider the scenario of a network connection, say a TCP connection. An object can be in various states; for example, a connection might already be established, a connection might be closed, or the object has already started listening through the connection. When this connection receives a request from other objects, it responds as per its present state.

The functionalities of a traffic signal or a television can also be considered to be an example of the State pattern. For example, you can change the channel if the TV is already in switched-on mode. It will not respond to the channel change requests if it is in switched-off mode.

© Vaskaran Sarcar 2018
V. Sarcar, *Design Patterns in C#*, https://doi.org/10.1007/978-1-4842-3640-6_20

www.EBooksWorld.ir

Computer World Example

Suppose you have a job-processing system that can process a certain number of jobs at a time. When a new job appears, either the system processes the job or it signals that the system is busy with the maximum number of jobs that it can process at a time. In other words, the system will send a busy signal if it gets another job-processing request when its total number of job-processing capabilities has been reached.

Illustration

This example models the functionalities related to a television and its remote control. Suppose you have a remote control to support the operations of a TV. You can simply assume that at any given point in time your TV can be in either of these three states: On, Off, or Mute. Initially the TV is in the Off state. When you press the On button on a remote control, the TV will be in the On state, and then if you press the Mute button, it will go into the Mute state. You can also assume that if you press the Off button when the TV is already in the Off state, if you press the On button when the TV is already in the On state, or if you press then Mute button when the TV is already in Mute mode, there will be no state change for the TV. The TV can go into the Off state from the On state or the Mute state (if you press the Off button). Figure 20-1 shows the state diagram that reflects all these possible scenarios.

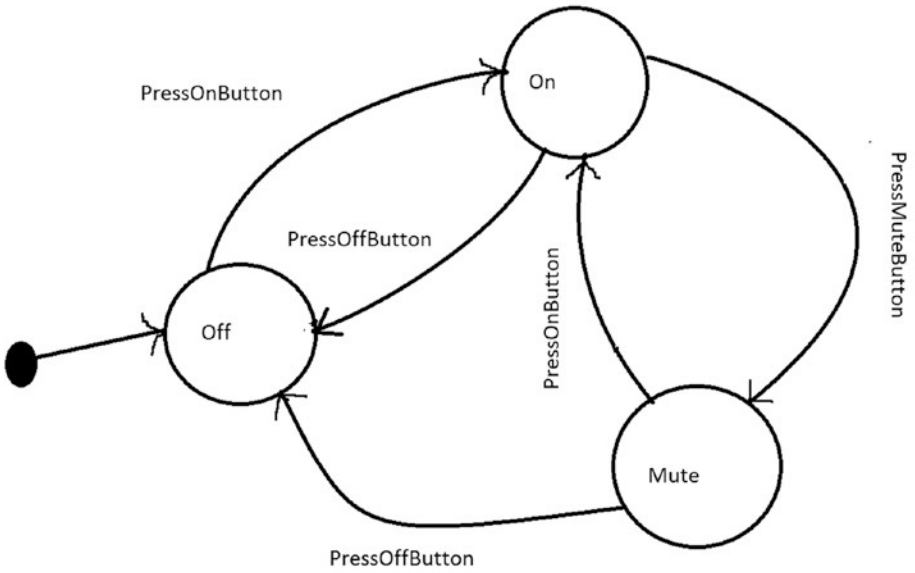


Figure 20-1. *Different states of a Television (TV)*

www.EBooksWorld.ir

For simplicity, in this diagram, I have not marked any state as final, though in the following illustration, you are switching off the TV at the end.

Class Diagram

Figure 20-2 shows the class diagram.

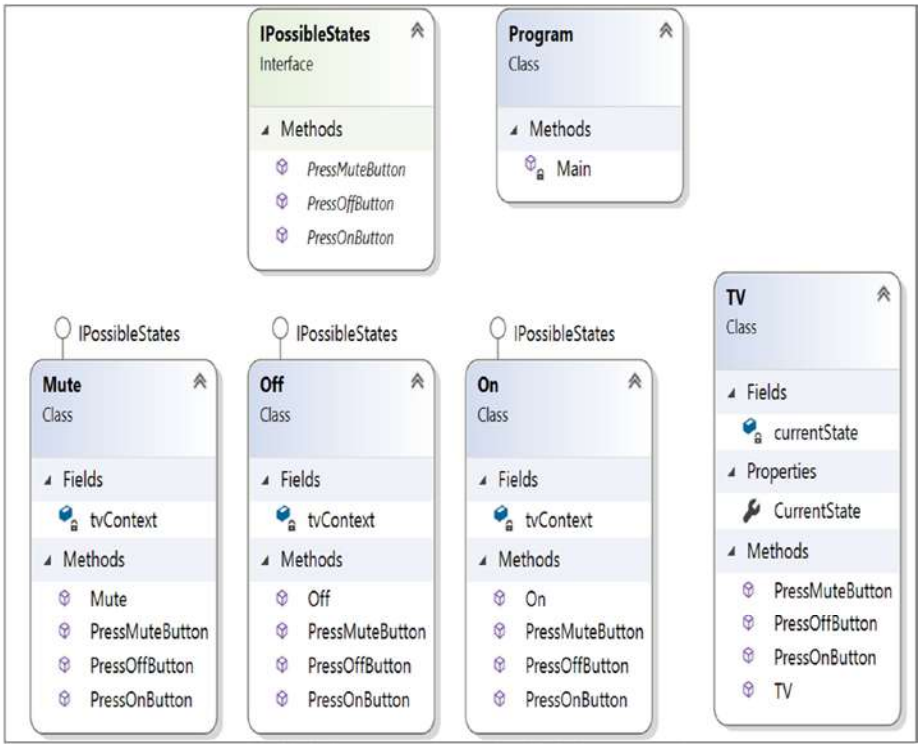


Figure 20-2. *Class diagram*

www.EBooksWorld.ir

Directed Graph Document

Figure 20-3 shows the directed graph document.

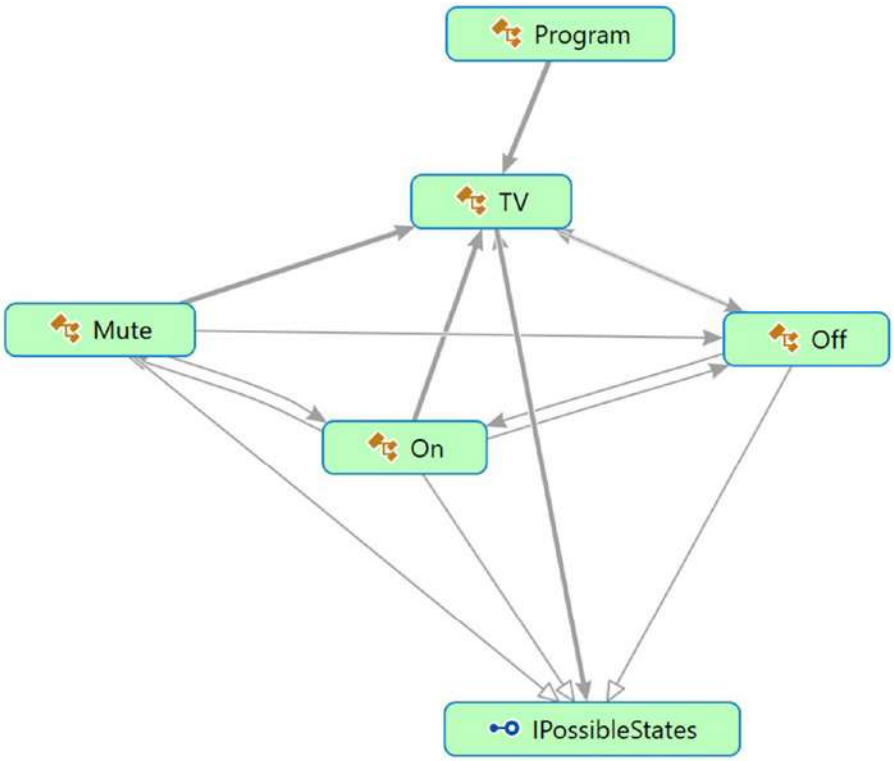


Figure 20-3. *Directed Graph Document*

Solution Explorer View

Figure 20-4 shows the high-level structure of the parts of the program.

www.EBooksWorld.ir

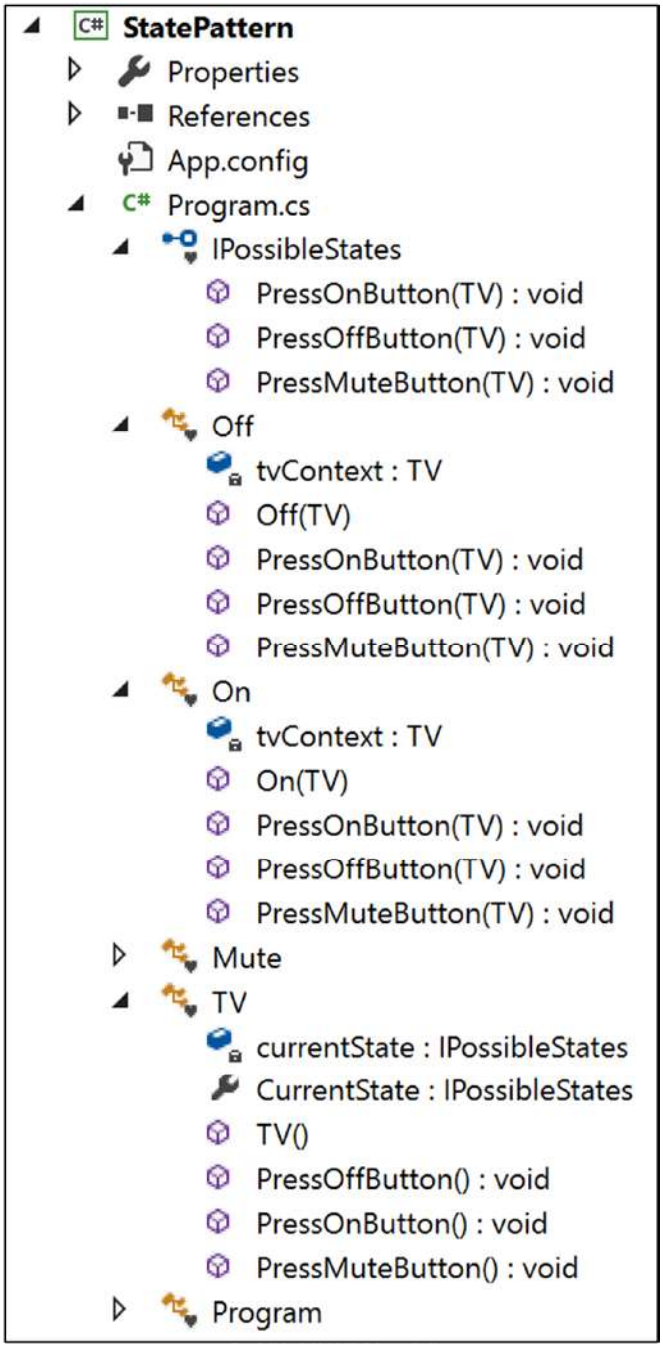


Figure 20-4. *Solution Explorer View*

www.EBooksWorld.ir

Implementation

Here's the implementation:

```
using System;
namespace StatePattern
{
   interface IPossibleStates
    {
        void PressOnButton(TV context);
        void PressOffButton(TV context);
        void PressMuteButton(TV context);
    }

    class Off : IPossibleStates
    {
        TV tvContext;
        //Initially we'll start from Off state
        public Off(TV context)
        {
            Console.WriteLine(" TV is Off now.");
            this.tvContext = context;
        }
        //Users can press any of these buttons at this state-On, Off or Mute
        //TV is Off now, user is pressing On button
        public void PressOnButton(TV context)
        {
            Console.WriteLine("You pressed On button. Going from Off to On
            state");
            tvContext.CurrentState = new On(context);
        }
        //TV is Off already, user is pressing Off button again
        public void PressOffButton(TV context)
        {
            Console.WriteLine("You pressed Off button. TV is already in
            Off state");
        }
```


www.EBooksWorld.ir

```
    //TV is Off now, user is pressing Mute button
    public void PressMuteButton(TV context)
    {
        Console.WriteLine("You pressed Mute button. TV is already in
        Off state, so Mute operation will not work.");
    }
}
class On : IPossibleStates
{
    TV tvContext;
    public On(TV context)
    {
        Console.WriteLine("TV is On now.");
        this.tvContext = context;
    }
    //Users can press any of these buttons at this state-On, Off or Mute
    //TV is On already, user is pressing On button again
    public void PressOnButton(TV context)
    {
        Console.WriteLine("You pressed On button. TV is already in On
        state.");
    }
    //TV is On now, user is pressing Off button
    public void PressOffButton(TV context)
    {
        Console.WriteLine("You pressed Off button. Going from On to Off
        state.");
        tvContext.CurrentState = new Off(context);
    }
    //TV is On now, user is pressing Mute button
    public void PressMuteButton(TV context)
    {
        Console.WriteLine("You pressed Mute button. Going from On to
        Mute mode.");
```


www.EBooksWorld.ir

```
            tvContext.CurrentState = new Mute(context);
        }
    }
    class Mute : IPossibleStates
    {
        TV tvContext;
        public Mute(TV context)
        {
            Console.WriteLine("TV is in Mute mode now.");
            this.tvContext = context;
        }
        //Users can press any of these buttons at this state-On, Off or Mute
        //TV is in mute, user is pressing On button
        public void PressOnButton(TV context)
        {
            Console.WriteLine("You pressed On button. Going from Mute mode
            to On state.");
            tvContext.CurrentState = new On(context);
        }
        //TV is in mute, user is pressing Off button
        public void PressOffButton(TV context)
        {
            Console.WriteLine("You pressed Off button. Going to Mute mode
            to Off state.");
            tvContext.CurrentState = new Off(context);
        }
        //TV is in mute already, user is pressing mute button again
        public void PressMuteButton(TV context)
        {
            Console.WriteLine(" You pressed Mute button. TV is already in
            Mute mode.");
        }
    }
```


www.EBooksWorld.ir

```
class TV
{
    private IPossibleStates currentState;
    public IPossibleStates CurrentState
    {
        //get;set;//Not working as expected
        get
        {
            return currentState;
        }
        /*Usually this value will be set by the class that
          implements the interface "IPossibleStates"*/
        set
        {
            currentState = value;
        }
    }
    public TV()
    {
        this.currentState = new Off(this);
    }
    public void PressOffButton()
    {
        currentState.PressOffButton(this);//Delegating the state
    }
    public void PressOnButton()
    {
        currentState.PressOnButton(this);//Delegating the state
    }
    public void PressMuteButton()
    {
        currentState.PressMuteButton(this);//Delegating the state
    }
}
```


www.EBooksWorld.ir

```
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("***State Pattern Demo***\n");
            //Initially TV is Off
            TV tv = new TV();
            Console.WriteLine("User is pressing buttons in the following
            sequence:");
            Console.WriteLine("Off->Mute->On->On->Mute->Mute->Off\n");
            //TV is already in Off state
            tv.PressOffButton();
            //TV is already in Off state, still pressing the Mute button
            tv.PressMuteButton();
            //Making the TV on
            tv.PressOnButton();
            //TV is already in On state, pressing On button again
            tv.PressOnButton();
            //Putting the TV in Mute mode
            tv.PressMuteButton();
            //TV is already in Mute, pressing Mute button again
            tv.PressMuteButton();
            //Making the TV off
            tv.PressOffButton();
            // Wait for user
            Console.Read();
        }
    }
}
```


www.EBooksWorld.ir

Output

Here's the output:

```
***State Pattern Demo***

 TV is Off now.
User is pressing buttons in the following sequence:
Off->Mute->On->On->Mute->Mute->Off

 You pressed Off button. TV is already in Off state
 You pressed Mute button. TV is already in Off state, so Mute operation
 will not work.
 You pressed On button. Going from Off to On state
 TV is On now.
You pressed On button. TV is already in On state.
You pressed Mute button. Going from On to Mute mode.
 TV is in Mute mode now.
 You pressed Mute button. TV is already in Mute mode.
You pressed Off button. Going to Mute mode to Off state.
TV is Off now.
```

Q&A Session

1. **Can you elaborate on how this pattern works in a real-world scenario?**

   Answer:

   Psychologists have repeatedly documented the fact that human beings can perform their best when they are in a relaxed mood but in the reverse scenario, when their minds are filled with tension, they cannot produce great results. That is why they always suggest that we work in a relaxed mood. Now consider a scenario. Suppose, you want to watch the live telecast of the winning moments of your favorite team. To watch and enjoy the moment, you need to power on the TV first. If the TV is not functioning properly at that moment and cannot be in "On" state, you cannot

www.EBooksWorld.ir

enjoy the moment. So, if you want to enjoy the moment through your TV, the first criteria is that the TV should change its state from “off “to “on”. The State pattern is helpful if you want to design a similar kind of behavior change in an object when its internal state changes.

2. **In this example, you have considered only three states of a TV: On, Off, and Mute. There can be many other states; for example, there may be a state that deals with connection issues or display conditions. Why have you ignored those issues?**

   Answer:

   The straightforward answer is because of simplicity. If the number of states increases significantly in the system, then it becomes difficult to maintain the system (and this is one of the key challenges associated with this design pattern). But if you understand this implementation, you can easily add any states you want.

3. **I have noticed that the GoF represented a similar structure for both the State pattern and the Strategy pattern in their famous book. I am confused by that.**

   Answer:

   Yes, the structures are similar, but you need to remember that their intents are different. Apart from this key distinction, you can think of the patterns like this: With a Strategy pattern, you are getting a better alternative to subclassing. In a State design pattern, different types of behaviors can be encapsulated in a state object, and the context is delegated to any of these states. When a context’s internal state changes, its behavior also changes.

   State patterns can also help you avoid lots of `if` conditions in some particular contexts. For example, consider the example once again.

www.EBooksWorld.ir

If the TV is in the Off state, it cannot go into the Mute state. From this state, it can move to the On state only. So, if you do not like the State design pattern, you may need to write the code like this:

```
class TV
{
//Some code before
public void PressOnButton()
{
if(currentState==OFF )
{
Console.WriteLine("You pressed On button. Going from Off to
OnState");
}
if(currentState==OFF )
{
Console.WriteLine("You pressed Off button. TV is already in Off
state");
}
else
{
Console.WriteLine("You pressed Mute button. TV is already in Off
state, so Mute operation will not work.");
}
//Some code after
}
```

4. **How are you implementing the open/close principle in your example?**

   Answer:

   Each of these TV states is closed for modification, but you can add a new state to the TV class.

www.EBooksWorld.ir

5. **What are the common characteristics between the Strategy pattern and the State pattern?**

   Answer:

   Both promote composition and delegation.

6. **It appears to me that these state objects are acting like singletons. Is this understanding correct?**

   Answer:

   Yes. Most of the times they act in this way.

7. **Can you avoid the use of contexts in the method parameters? For example, can you avoid them in statements like this?**

   ```
   void PressOnButton(TV context);
   ```

   Answer:

   No. Through them you are saving states. They are helping you to evaluate whether you are changing between states or you are already in the same state. Notice the output also. These contexts are helping you to get output like this: "You pressed Mute button. TV is already in Mute mode."

8. **What are the pros and cons of the State design pattern?**

   Answer:

   The pros are as follows:

   - You have already seen that by following the open/close principle, you can add new states and extend a state's behavior easily.
   - It reduces the use of `if-else` statements. In other words, the conditional complexity is reduced. (Refer to the answer to question 3 earlier.)

   You probably already noticed that having more states means having more scattered code, and the obvious side effect is difficult maintenance for you.

www.EBooksWorld.ir